# User-defined button combos in SuperCollider

The holdwithtrigg feature for SuperCollider instruments on QB Nebulae V2. Hold one button, press another, and a separate trigger (user_defined_1 .. user_defined_7) is sent to SuperCollider instead of the two buttons' normal actions.

## 1. What it does

Example from MiClouds.scd: hold File, then press Reset. SuperCollider receives one pulse on user_defined_1, which MiClouds uses as record_reset. File does not change file and Reset does not send its normal reset.

- The hold button must be physically pressed (raw button state = 1).
- The trigger channel must get a rising edge (0 -> 1) while the hold button is down.
- When both are true, user_defined_N is set to 1 and both buttons are marked ignore_until_release.
- user_defined_N goes back to 0 after the pulse length (see section 4).

## 2. Why ignore_until_release is needed

ignore_until_release sits on the two physical buttons in the combo (File and Reset), not on user_defined_1..7. Those are plain values in a dict in controlhandler.py, with their own pulse timer.

Without the flag, one combo would give user_defined_1 AND the ordinary actions: Reset's trigger, and a file change when File is released (File is incremental,falling, so it acts on release). With the flag, only user_defined_N reaches SuperCollider. The flag is cleared when the button has been released for two reads in a row.

Limits of the flag:

- It can only block what happens after the combo is detected. Use a falling-edge button as the hold button. A rising-edge button would already have done its normal action when you pressed it.
- Order matters: hold first, then press the trigger button. Pressing the trigger button first gives its normal action and no combo.
- Blocking only happens in the SuperCollider branch of updateAll(). sendScOscMessages() sends all channels without checking the flag.

## 3. Instructions: what to do in your .scd file

Step 1 - add a holdwithtrigg line to the nebconfig block at the top of the .scd file:

```
<hold_button>,holdwithtrigg,<trigger_button>,trigger,<user_defined_N>
```

| Field | Meaning |
|---|---|
| `<hold_button>` | Button you hold down, e.g. file. Should be a digital, falling-edge button. |
| `holdwithtrigg` | Keyword that marks the line as a combo. |
| `<trigger_button>` | Button you press while holding, e.g. reset. |
| `trigger` | Action. Parsed but not used yet - every combo sends a pulse. Write trigger. |
| `<user_defined_N>` | Output: user_defined_1 .. user_defined_7. |

Keep the normal line for the hold button as well (e.g. file,incremental,falling). Combo lines are stored in a separate list, so they do not replace the button's own config. You can have several combo lines, one per output.

Step 2 - read the output in your SynthDef with the matching UGen (NebUserDefined1 .. NebUserDefined7):

```
var record_reset = NebUserDefined1.kr(0, 1);   // 0 normally, 1 during the
pulse
```

Step 3 - use it as a trigger. The value stays at 1 for the whole pulse, so use an input that triggers on 0 -> positive, or make a single-sample trigger yourself:

```
var t = Trig1.kr(NebUserDefined1.kr(0, 1) > 0.5, ControlDur.ir);
```

## Complete example (based on MiClouds.scd)

```
/*
nebconfigbegin
ksmps,64
sr,48000
reset,triggered,rising
file,incremental,falling
file,holdwithtrigg,reset,trigger,user_defined_1
nebconfigend
*/

s.waitForBoot {
    NebInterface.init(s);
    s.sync;
    SynthDef(\myinstr, {
        var file  = NebFile.kr(0, 1);
        var reset = NebReset.kr(0, 1);
        var combo = NebUserDefined1.kr(0, 1);   // File held + Reset
        // ... use combo as a trigger input ...
    }).add;
    s.sync;
    ~synth = Synth(\myinstr);
    NebInterface.ready(s);
};
```

## 4. Pulse length

The pulse length is output_period, computed in instrparser.getOutputPeriod():

```
output_period = max(100 ms, (ksmps / sr) * 1000 * 75)
```

With ksmps 64 and sr 48000 (MiClouds) this gives 100 ms.

## 5. Where it lives in the code

| File | What it does |
|---|---|
| `instrparser.py` | populateDict() collects holdwithtrigg lines into holdWithTriggList; getHoldWithTriggCombos() returns them. |
| `nebulae.py` | Passes the combos to the SC control handler as scConfigData['holdwithtrigg_combos']. |
| `controlhandler.py` | checkHoldWithTrigg() detects the combo, sets user_defined_N and the ignore flags. updateAll() (SC branch) resets pulses, skips ignored channels and sends user_defined_1..7 over OSC. |

| | |
|---|---|
| `digitaldata.py` | ignore_until_release: the button keeps its previous state until released. |
| `control.py` | Passes setIgnoredUntilReleased() / isIgnoredUntilReleased() to the digital input. |
| `NebInterface.sc` | Buses \user_defined_1..7 and UGens NebUserDefined1..7. |

## 6. Open points in the code

- The action field (trigger) is parsed but never used.
- checkHoldWithTrigg() returns a suppress set that nothing reads.
- sendScOscMessages() does not call checkHoldWithTrigg() and does not skip ignored channels.
- neb_globals.py has wifi_always_on = True uncommitted - probably a debug setting, not part of this feature.